
When to End, Which to Invoke Next: Transition-Aware Tokenized Procedural Memory for LLMs

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 *One-Token Procedural Memory (TokMem)* has recently emerged as a parameter-
2 efficient paradigm for storing and reusing task-solving procedures in large language
3 models (LLMs), encoding each procedure as a compact memory token. This paper
4 studies a more challenging regime: *an LLM must reliably orchestrate a sequence*
5 *of consecutive procedure recalls within a single generation*. We identify a **fundamental**
6 **limitation** of TokMem-style procedure chaining: memory tokens provide
7 explicit entry points into procedures, but the transition from one procedure to the
8 next remains implicit. This under-considered handoff makes next-procedure selection
9 increasingly fragile as procedural chains grow longer. To address this issue, we
10 propose *Transition-Aware Procedural Memory (TapMem)*, a lightweight framework
11 that makes inter-procedure handoffs explicit. The key idea behind TapMem is to
12 model each inter-procedure handoff as a boundary-conditioned selection problem.
13 Since the next procedure should be selected only after the current procedure-guided
14 span ends, TapMem first introduces **Explicit Termination Signals** to make this
15 completion boundary observable. Building on the resulting boundary representation,
16 TapMem further employs a **Transition-Conditioned Routing Adapter** to
17 produce a context-dependent correction over candidate memory-token logits. By intervening
18 only at transition positions and only over memory-token logits, TapMem
19 preserves full-vocabulary autoregressive generation while strengthening the sparse
20 procedural decisions required for reliable multi-step orchestration. Comprehensive
21 experiments across diverse procedure-recall settings demonstrate that TapMem
22 achieves promising performance, yielding average gains of 5.37% in tool selection
23 and 5.03% in argument generation.

1 Introduction

25 Large language models (LLMs) [36, 37] are increasingly deployed in tasks where success depends not
26 only on generating fluent text, but also on faithfully following a prescribed task-solving procedure [9,
27 13]. For instance, a model may need to decompose a request, call an external operation, reason over
28 intermediate results, verify partial outputs, and finally format the response in a required way [25, 49,
29 7, 14]. These behaviors are inherently procedural: they specify how the model should act under a
30 given context, rather than what factual content should be retrieved. A broad body of related work has
31 studied procedural memory, which is discussed in Appendix A. As LLM-based applications become
32 more structured [11, 28, 32, 1, 5], a key challenge is to represent reusable procedures compactly and,
33 more importantly, to compose them reliably within a single generation.

34 Tokenized procedural memory offers a promising paradigm for this goal. In its typical formulation,
35 One-Token Procedural Memory (TokMem) [44] encodes each reusable procedure as a compact
36 memory token, thereby casting *procedural recall as a language-modeling problem*: invoking a

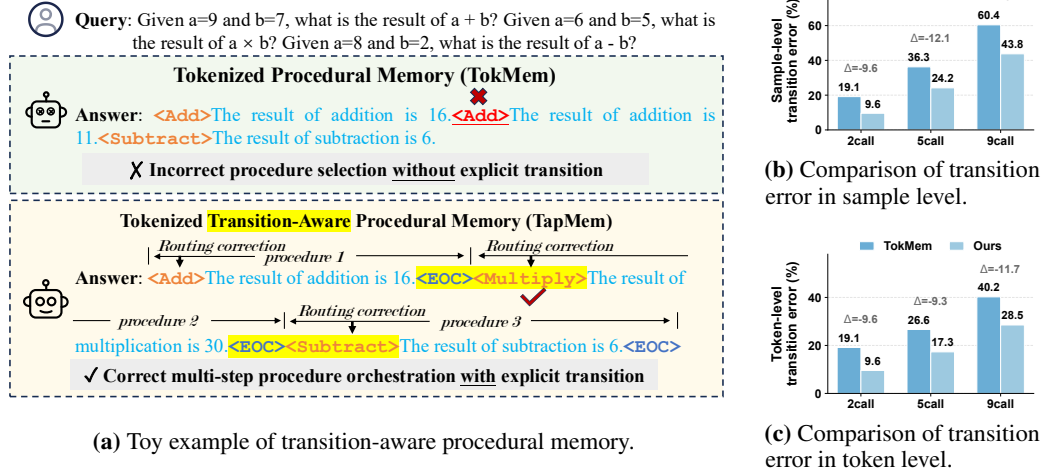


Figure 1: (a) A toy example of *TokMem* and *TapMem* (Ours). *TokMem* incorrectly selects the second procedure as <Add>, whereas *TapMem* correctly selects the second procedure as <Multiply> based on the <EOC> token and routing correction. (b)(c) Compared with *TokMem*, our method achieves a larger reduction in sample-level and token-level transition errors, and this advantage grows as the number of calls increases. More details can be found in Appendix D.2.

procedure amounts to predicting its corresponding memory token. This design enables parameter-efficient invocation of individual procedures, but compositional recall exposes an inherent limitation: the next memory token must implicitly act both as the boundary signal for the current procedure and as the selector of the next one, weakening compositional reliability as procedure chains grow longer. Figure 1(a) provides a toy example of this failure mode, where *TokMem* selects the first procedure correctly but fails at the subsequent transition. Figure 1(b)-(c) further show that its transition errors become sharp with longer procedure chains, motivating explicit boundary modeling for reliable next-procedure selection. This leads to our central question:

How to develop a more reliable framework for multi-step procedural orchestration?

To answer this question, we propose *Transition-Aware Procedural Memory (TapMem)*, a lightweight framework that augments tokenized procedural memory with an explicit transition mechanism. The key observation is that **a transition between two procedures contains an inherent order**: the LLM must first recognize that the current procedure-guided span has ended, and only then should it select the next procedure memory. *TapMem* models each inter-procedure transition into two explicit and trainable stages [10, 2, 34]. Specifically, *TapMem* first introduces Explicit Termination Signals to turn the end of a procedure-guided span into a supervised boundary event, rather than leaving it to be inferred from ordinary autoregressive continuation. Subsequently, a Transition-Conditioned Routing Adapter is proposed on top of the exposed boundary state to correct candidate memory-token logits for next-procedure selection. Most importantly, This boundary-conditioned correction acts only at transition positions and only over memory-token logits, preserving full-vocabulary autoregressive generation while strengthening the sparse procedural decisions required for reliable multi-step orchestration. Experiments on compositional and atomic procedure recall show consistent gains over *TokMem*, with larger improvements on longer procedure chains.

Our main contributions are summarized as follows:

- **We reveal an overlooked transition bottleneck in tokenized procedural memory.** Existing *TokMem*-style methods assign each procedure a dedicated memory token, but leave inter-procedure transitions implicit, entangling procedure termination with next-procedure selection in autoregressive prediction.
- **We introduce *TapMem* for transition-aware procedure memory of LLMs.** *TapMem* separates procedure completion from next-procedure selection with Explicit Termination Signals

and a Transition-Conditioned Routing Adapter, while preserving full-vocabulary autoregressive generation.

- **We validate the effectiveness of TapMem across atomic and compositional recall.** Results and ablations show consistent gains over TokMem, demonstrating that explicit boundary exposure and transition-conditioned routing jointly improve the reliability of tokenized procedural memory.

2 Methodology

In this section, we briefly introduce the problem of tokenized procedural memory and present our TapMem method. **Related work is deferred in Appendix A due to space limitations.**

2.1 Problem Formulation & Motivation

This paper studies *tokenized procedural memory* for multi-step procedure orchestration in large language models (LLMs). Following prior work on TokMem [44], a *procedure is defined as* a reusable context-response or task-solving routine that specifies how a model should act in a given context, such as task decomposition, tool invocation, and response formatting. To enable LLMs to manage and reuse such procedures efficiently, TokMem represents each reusable procedure with a compact memory token, which serves as a learned handle for eliciting the corresponding behavior from the language model. Formally, let f_θ denote a pretrained LLM with the original vocabulary $\mathcal{V}$, where θ denotes the backbone parameters. TokMem first augments the original vocabulary with K special procedural memory tokens:

$$\tilde{\mathcal{V}} = \mathcal{V} \cup \mathcal{V}_M, \quad \mathcal{V}_M = \{m_1, m_2, \dots, m_K\}, \quad \mathcal{V} \cap \mathcal{V}_M = \emptyset. \quad (1)$$

here, $\mathcal{V}_M$ is not a separate output space, but an extension of the original language-model vocabulary. Each memory token $m_i \in \mathcal{V}_M$ is assigned a new token index in the augmented vocabulary $\tilde{\mathcal{V}}$ and is associated with a trainable embedding vector $\mathbf{e}_i \in \mathbb{R}^d$, where d is the embedding dimension. The trainable embeddings of all procedural memory tokens form a memory bank:

$$\mathbf{E} = \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_K^\top \end{bmatrix} \in \mathbb{R}^{K \times d}. \quad (2)$$

Given an input query, the goal is to generate a response by selecting and invoking **one or more appropriate procedures** indexed by $\mathcal{V}_M$. Suppose that a training instance with $J \geq 1$ procedure calls is serialized as an autoregressive sequence:

$$\mathbf{y} = [\mathbf{q}, m^{(1)}, \mathbf{a}^{(1)}, m^{(2)}, \mathbf{a}^{(2)}, \dots, m^{(J)}, \mathbf{a}^{(J)}], \quad (3)$$

where $\mathbf{q}$ denotes the tokenized query, $m^{(j)} \in \mathcal{V}_M$ is the memory token selected for the j -th procedure call, and $\mathbf{a}^{(j)} = (a_1^{(j)}, \dots, a_{L_j}^{(j)})$ denotes the ordinary response segment generated under that procedure. Specifically, we denote $\mathbf{y}_0 = \mathbf{q}$. TokMem optimizes the trainable memory bank $\mathbf{E}$ using the standard autoregressive next-token prediction objective:

$$\mathcal{L}_{\text{AR}}(\mathbf{y}; \mathbf{E}) = - \sum_{r=|\mathbf{q}|+1}^{|\mathbf{y}|} \log p_{\theta, \mathbf{E}}(y_r \mid y_{<r}), \quad (4)$$

where $|\mathbf{q}| + 1$ denotes the position of the first generated token, r denotes the index of token in the sequence.

Discussions The objective in Eq. (4) provides a compact and parameter-efficient mechanism for procedure recall. However, we argue that it leaves a key part of compositional procedure orchestration under-modeled. From the perspective of sequence serialization, procedural memory tokens $m^{(j)}$ and ordinary response tokens $a_r^{(j)}$ are optimized under the same next-token prediction objective in Eq. (4).

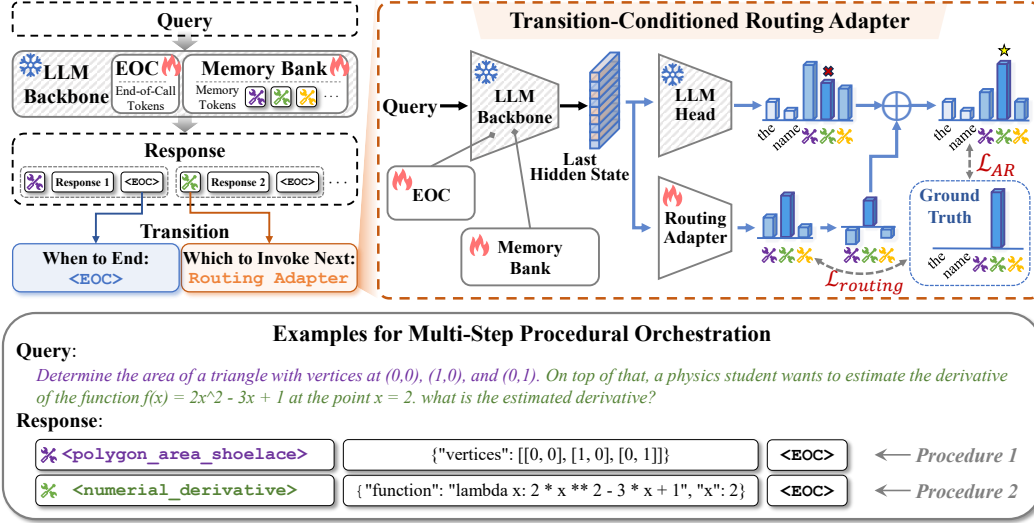


Figure 2: **Overview of our TapMem approach.** Given a query, the model selects a memory token from the memory bank to determine the current procedure, then generates the corresponding response, and finally emits <EOC> to mark the end of this procedure. At positions corresponding to memory token generation, TCRA performs routing correction on the LM output logits. The bottom panel shows a case of multi-step procedural orchestration.

Let $\mathbf{y}_j$ denote the autoregressive history before the $j + 1$ -th procedure call. After $\mathbf{a}^{(j)}$, TokMem directly predicts the next memory token:

$$p_{\theta, \mathbf{E}}(m^{(j+1)} | \mathbf{y}_j), \quad j < J, \quad (5)$$

This formulation implicitly conflates two structurally distinct decisions: whether the current procedure has completed, and which procedure should be invoked next. Such an implicit transition mechanism may be sufficient for isolated procedure recall, but **becomes fragile when $J > 1$** , where reliable procedure chaining requires accurate procedure boundaries and transition dependencies across multiple procedure calls. To address this challenge, we therefore reformulate tokenized procedural memory as a *transition-aware procedure orchestration* problem, where procedure termination and next-procedure routing are explicitly modeled rather than left to generic autoregressive prediction.

2.2 TapMem: Transition-Aware Procedure Memory

To address the transition bottleneck identified in Section 2.1, we propose *Transition-Aware Procedural Memory (TapMem)*. TapMem is built on a simple structural principle: in a procedure chain, the next procedure should be selected only after the current procedure-guided span has been completed. Instead of letting the next memory token implicitly signal both completion and selection, TapMem explicitly exposes the completion boundary and performs memory-token routing at that boundary. As shown in Figure 2, TapMem realizes this principle with two components: an end-of-call token <EOC> that marks procedure ending, and a *Transition-Conditioned Routing Adapter* that corrects candidate memory-token logits at the exposed boundary state. This design preserves ordinary response generation inside each procedure while strengthening the sparse decisions required for reliable multi-step orchestration.

2.2.1 When to End: Explicit Termination Signals

The first question in transition-aware procedural memory is *when the current procedure should end*. In TokMem-style chaining, a memory token selects the current procedure, and the following response span is generated under that procedure. However, the completion of this procedure-guided span is not represented by a dedicated token-level event. Instead, it is only implied when the next memory token appears. As a result, the next memory token implicitly serves two roles: it indicates that the

current procedure has completed and selects the next procedure to invoke. This coupling makes the boundary between consecutive procedures under-specified, especially when multiple procedures must be chained within a single generation.

To expose this boundary, TapMem introduces an end-of-call token $\langle \text{EOC} \rangle$ as an *Explicit Termination Signal*. The role of $\langle \text{EOC} \rangle$ is to turn procedure completion into an explicit autoregressive prediction target. By inserting $\langle \text{EOC} \rangle$ after each procedure-guided span, the model receives direct token-level supervision for where the current procedure call ends, rather than learning this boundary only indirectly from the appearance of the next memory token. Unlike ordinary textual delimiters or $\langle \text{EOS} \rangle$, $\langle \text{EOC} \rangle$ is a reserved procedural marker: it denotes the completion of the current procedure call without terminating the whole output sequence. Thus, $\langle \text{EOC} \rangle$ separates procedure-level completion from sequence-level termination while remaining compatible with standard autoregressive decoding.

Formally, a single procedure call is serialized as

$$m, \mathbf{a}, \langle \text{EOC} \rangle, \quad (6)$$

where $m \in \mathcal{V}_M$ is the memory token that selects the current procedure, and $\mathbf{a}$ denotes the response span generated under that procedure. For a multi-step trajectory with J procedure calls, TapMem constructs the target sequence as

$$\mathbf{y}' = [\mathbf{q}, m^{(1)}, \mathbf{a}^{(1)}, \langle \text{EOC} \rangle, \dots, m^{(J)}, \mathbf{a}^{(J)}, \langle \text{EOC} \rangle], \quad (7)$$

where $\mathbf{q}$ is the input query, $m^{(j)}$ is the memory token for the j -th procedure, and $\mathbf{a}^{(j)}$ is the corresponding procedure-guided response span. Once $\langle \text{EOC} \rangle$ is generated, the following position becomes a natural boundary for selecting the next procedure memory if the chain continues. We therefore collect the procedure-selection boundary positions as

$$\mathcal{B} = \{|\mathbf{q}| + 1\} \cup \{b \in \{|\mathbf{q}| + 2, \dots, |\mathbf{y}'|\} \mid y'_{b-1} = \langle \text{EOC} \rangle, y'_b \in \mathcal{V}_M\}. \quad (8)$$

The first term corresponds to the initial procedure selection after the query. The remaining elements correspond to positions immediately following $\langle \text{EOC} \rangle$ where the target token is the memory token for the next procedure. Positions outside $\mathcal{B}$ correspond to ordinary response generation or the prediction of $\langle \text{EOC} \rangle$ itself. In this way, $\langle \text{EOC} \rangle$ exposes procedure completion as a learnable boundary event while preserving the standard generation behavior of the underlying LLM. These boundary positions provide the explicit decision points at which the model should select the next procedure memory, motivating the transition-conditioned routing mechanism introduced next.

2.2.2 Which to Invoke Next: Transition-Conditioned Routing Adapter

After $\langle \text{EOC} \rangle$ explicitly marks the end of the current procedure, the next question is *which procedure should be invoked next*. Although the original LM head can predict the next memory token, we argue that such a decision should be different from those ordinary response generation: At a procedure-selection position, the model should primarily distinguish among candidate procedure memories, while natural-language and argument generation should remain governed by the base LM. Therefore, in order to strengthen memory-token selection without disturbing ordinary generation, we propose the *Transition-Conditioned Routing Adapter* (TCRA).

As shown in Fig. 2, TCRA provides a routing signal only at procedure-selection positions, where the model needs to choose among candidate procedure memories rather than continue ordinary response generation. Let $h_r \in \mathbb{R}^d$ denote the hidden state of the language model at a boundary position $r \in \mathcal{B}$. TCRA reads this boundary representation and maps it to routing logits over candidate procedure memories:

$$z_r = g_\phi(h_r) \in \mathbb{R}^K, \quad (9)$$

where g_ϕ is a lightweight projection adapter and K is the number of memory tokens. The logits z_r encode a transition-conditioned preference over candidate procedure memories, which is later injected only into memory-token logits, so ordinary vocabulary logits remain governed by the base LM head.

We then convert this routing preference into a reference-normalized logit adjustment over memory-token logits. Let $\pi_r = \text{softmax}(z_r)$. For each memory token m_k , we define the adjustment as

$$\Delta_r(k) = \alpha (\log \pi_r(k) - \log(1/K)), \quad (10)$$

where $1/K$ is the uniform reference distribution and α controls the adjustment strength. The uniform reference defines a zero-adjustment point: if the adapter **assigns no preference among candidate memories**, i.e., $\pi_r(k) = 1/K$, then $\Delta_r(k) = 0$. In this way, TCRA modifies memory-token logits only when the boundary representation provides a non-uniform preference over candidate procedure memories.

Taking a step further, let $\ell_r^{\text{LM}} \in \mathbb{R}^{|\tilde{\mathcal{V}}|}$ denote the base language-model logits over the augmented vocabulary. At a procedure-selection position r , TapMem applies the adjustment only to procedural memory-token logits:

$$\tilde{\ell}_r(v) = \begin{cases} \ell_r^{\text{LM}}(v) + \Delta_r(k), & v = m_k, m_k \in \mathcal{V}_M, \\ \ell_r^{\text{LM}}(v), & \text{otherwise.} \end{cases} \quad (11)$$

For positions outside $\mathcal{B}$, TapMem directly uses the original language-model logits without adjustment. Therefore, TCRA strengthens next-procedure selection only at procedure-selection positions, while preserving ordinary full-vocabulary generation inside each procedure-guided span.

2.3 Training and Inference with TapMem

TapMem follows the autoregressive objective $\mathcal{L}_{\text{AR}}$ defined above, but uses TCRA-adjusted logits at procedure-selection boundaries. Specifically, for $b \in \mathcal{B}$, TCRA adjusts only candidate memory-token logits according to Eq. 11; at all other positions, the original language-model logits are used unchanged. This aligns training with inference, where the same boundary-time adjustment is applied during decoding. Since next-procedure selection occurs only at sparse boundary positions, $\mathcal{L}_{\text{AR}}$ provides only indirect supervision for the routing adapter. We therefore add a supervised routing loss over procedure-selection boundaries:

$$\mathcal{L}_{\text{route}} = -\frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \log \pi_b(m_b),$$

where $m_b \in \mathcal{V}_M$ denotes the target procedure memory at boundary b . The full objective is

$$\mathcal{L} = \mathcal{L}_{\text{AR}} + \gamma \mathcal{L}_{\text{route}},$$

where γ controls the strength of the routing supervision.

During training, the backbone LLM remains frozen, and we update only the added memory-token embeddings, the `<EOC>` embedding, and the routing adapter g_ϕ . The routing branch reads boundary hidden states with stop-gradient. This design lets $\mathcal{L}_{\text{route}}$ train the adapter to improve next-procedure selection, without directly reshaping hidden representations used for ordinary response generation. Our ablation experiments 3.4 demonstrate the effectiveness of this design.

During inference, TapMem applies TCRA only at procedure-selection positions, i.e., the initial generation position and positions immediately following `<EOC>`. All other tokens are decoded with the original LM logits, preserving ordinary autoregressive generation. Detailed algorithms are provided in Appendix C.

3 Experiments

In this section, we describe the details of the experiments and present our results. Due to space limitations, please refer to the Appendix D and E for an extended version.

3.1 Experiments Setup

Implementation Details. All experiments are implemented on a single NVIDIA-A100 80GB GPU using mixed-precision bfloat16 training. In the main experiments, TokMem [44] serves as the backbone of tokenized procedural memory, and we use a linear projection as the adapter. The training is conducted on synthetic data from APIGen [26] for compositional recall and on the SNI [39] dataset for atomic recall. We adopt *Adam with Weight Decay (AdamW)* [19, 27] optimizer with a weight decay of 0.01. The backbone models remain frozen during training. We fine-tune LoRA [16] with a rank of 8. In TokMem, only the embeddings of the newly added tokens are trained. In TapMem, both the added token embeddings and the additional linear routing adapter are trained.

Table 1: **Compositional recall results on the APIGen dataset in terms of F1.** The champion and the runner-up are highlighted in **bold** and underline.

Method	Tool Selection				Argument			
	2 calls	3 calls	4 calls	Avg.	2 calls	3 calls	4 calls	Avg.
<i>Llama 3.2 1B</i>								
ICL	2.0	0.9	0.0	1.4	1.2	0.4	0.0	0.8
RAG	15.5	3.6	0.0	9.0	9.7	2.0	0.0	5.5
TokMem	82.0	80.9	76.0	<u>81.2</u>	67.6	66.0	55.7	<u>66.2</u>
Ours	89.8	90.3	83.3	89.7	74.8	74.6	65.6	74.3
Fine-Tuning	82.2	80.3	74.7	80.9	79.3	78.0	65.0	78.0
TokMem (w/ adapt)	97.9	98.3	94.9	<u>97.9</u>	83.3	83.6	75.4	<u>83.1</u>
Ours (w/ adapt)	98.1	98.2	97.3	98.2	85.9	84.6	77.9	84.9
<i>Llama 3.2 3B</i>								
ICL	59.5	61.2	56.9	60.2	48.7	49.4	45.2	48.9
RAG	80.2	77.0	81.5	78.7	55.2	52.8	47.3	53.7
TokMem	77.6	77.7	74.0	<u>77.4</u>	62.9	61.7	56.6	<u>62.0</u>
Ours	81.0	83.6	82.9	82.4	63.4	67.7	64.6	65.5
Fine-Tuning	97.3	98.7	92.4	97.7	87.9	87.3	77.8	87.1
TokMem (w/ adapt)	99.6	99.4	97.2	99.4	88.2	86.6	80.0	<u>87.0</u>
Ours (w/ adapt)	99.1	99.4	100.0	<u>99.3</u>	87.4	88.3	82.6	87.6
<i>Llama 3.1 8B</i>								
ICL	57.7	63.6	52.3	60.3	49.6	55.2	49.7	52.3
RAG	79.0	78.2	74.1	78.4	60.1	60.7	55.1	60.1
TokMem	84.3	85.1	72.7	<u>84.1</u>	72.7	70.7	62.5	<u>71.2</u>
Ours	87.9	86.4	78.5	86.7	76.5	73.7	68.3	74.7
Fine-Tuning	97.1	98.2	95.9	97.6	88.8	87.3	84.9	87.9
TokMem (w/ adapt)	97.8	98.7	98.1	<u>98.2</u>	87.1	87.6	82.3	87.1
Ours (w/ adapt)	99.5	98.9	95.7	99.0	87.9	88.0	79.6	<u>87.5</u>

Competitors. Our method improves procedural memory by modeling transitions explicitly. We compare it with other methods, including textual prompting methods (Base/ICL), retrieval-augmented textual memory (RAG [21]), parameter-efficient fine-tuning (LoRA [16]), and the tokenized procedural memory baseline TokMem [44]. Among them, Base/ICL and RAG store reusable procedure information in external text. LoRA stores it through lightweight parameter updates. TokMem encodes procedures as trainable memory tokens.

Evaluation Protocol. Following TokMem [44], we evaluate our method on *Compositional Recall* and *Atomic Recall*. For compositional recall, we use APIGen [26] and report *Tool Selection F1* and *Argument Prediction F1* on Llama 3.2 1B/3B/8B [36]. These metrics measure procedural memory selection and full function-call correctness. For atomic recall, we use SNI [39] and report *Routing Accuracy* and *ROUGE-L* on Qwen 0.5B [37] and Llama 3.2 3B [36]. Routing accuracy measures memory-token selection, and ROUGE-L [23] evaluates response quality. All methods use the same data splits, backbones, and metrics.

3.2 Overall Performance

Compositional Recall Task. Table 1 reports the compositional recall results on APIGen. The comparison with text-based baselines highlights the difficulty of this task. ICL and RAG improve with larger backbones, but they remain less stable than memory-based or parameter-updating methods, especially when the model must generate multiple procedure calls in sequence. Notably, ICL drops to 0 on 4-call tool selection for Llama 3.2 1B. Although TokMem achieves stronger performance through tokenized procedural memory, its performance also drops clearly as the number of tool calls increases. **In contrast, our TapMem achieves better performance in both tool selection and argument generation, and shows only a small drop as the number of calls increases.** TapMem yields average gains of 5.37% in tool selection and 5.03% in argument generation across all models. As the number of calls increases, our method consistently outperforms TokMem by 3.4 at least. On the 3B model, we even improve Tool Selection F1 from 77.4 to 82.4. This result suggests that explicit

Table 2: **Atomic recall results on the SNI dataset.** This setting contains a single procedural memory selection per example. Question Answering and Program Execution denote the two largest task categories in selected tasks. Avg. denotes the average performance across all tasks. The champion and the runner-up are highlighted in **bold** and underline.

Method	Routing Acc			Rouge-L		
	Question Answering	Program Execution	Avg.	Question Answering	Program Execution	Avg.
<i>Qwen 0.5B</i>						
Base	–	–	–	31.23	31.79	40.45
RAG	–	–	–	32.64	27.63	33.43
TokMem	75.64	92.44	87.31	33.21	31.27	42.10
Ours	80.83	94.37	89.04	33.51	33.31	42.50
<i>Llama 3.2 3B</i>						
Base	–	–	–	23.00	7.08	20.15
RAG	–	–	–	37.51	37.01	40.25
TokMem	84.51	84.47	86.94	45.46	44.29	48.85
Ours	90.85	84.77	87.60	50.53	45.09	49.47

Table 3: Ablation study on the effectiveness of <EOC> and TCRA in terms of F1.

<EOC>	TCRA	Tool Selection				Argument			
		2 calls	3 calls	4 calls	Avg.	2 calls	3 calls	4 calls	Avg.
Llama 3.2 1B									
		82.0	80.9	76.0	81.2	67.6	66.0	55.7	66.2
✓		84.4	85.7	76.0	84.6	70.7	72.0	57.0	70.7
✓	✓	89.8	90.3	83.3	89.7	74.8	74.6	65.6	74.3

modeling of procedure transitions helps the model select the next procedural memory more reliably, and that better routing further improves the quality of the full function-call output.

TokMem [44] shows that fine-tuning with additional data further improves model performance. This is because the original LLMs have not seen compositional recall tasks and therefore have limited generalization ability. Fine-tuning helps the model better understand this process. We also evaluate our method under this setting with adaptation. The results show that **TapMem remains effective, especially for smaller models with limited capacity**, where it improves to 98.2 after adaptation on Llama 3.2 1B. These gains suggest that transition-aware procedural memory remains useful even when the backbone has learned the basic compositional pattern.

Atomic Recall Task. To further evaluate performance in each procedure selection, we analyze single-step procedure selection, which we refer to as the atomic recall task. Unlike the compositional recall task, it requires only one procedural memory. Table 2 reports the performance comparison. The results show that TapMem also improves single-step procedure routing in this setting, where routing accuracy increases from 87.31 to 89.04 on Qwen 0.5B and from 86.94 to 87.60 on Llama 3.2 3B. The result further supports the effectiveness of TapMem: beyond modeling transitions in compositional trajectories, its routing refinement also makes isolated memory-token selection more reliable.

3.3 Compositional Recall with More Tool Calls

To evaluate performance with more tool calls, we extend compositional recall to settings with up to 10 calls. Figure 4 compares TokMem and our method under this more challenging setting. Our method consistently outperforms TokMem and keeps tool selection above 85 across all call counts, while the argument generation score of TokMem drops below 68 in the 10-call setting. On average, our method exceeds TokMem by 3.5 in tool selection and by 3.6 in argument generation. These results show that TapMem is both effective and stable in complex multi-step tool-calling settings and highlight the importance of multi-step procedural orchestration.

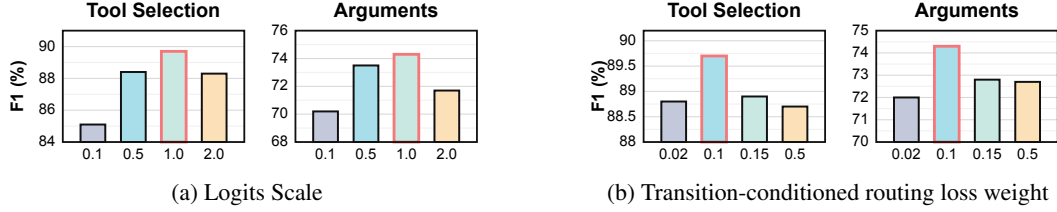


Figure 3: Ablation results on logits scale and transition-conditioned routing loss weight.

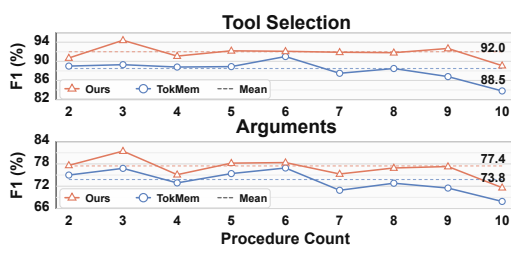


Figure 4: Fine-grained performance comparisons

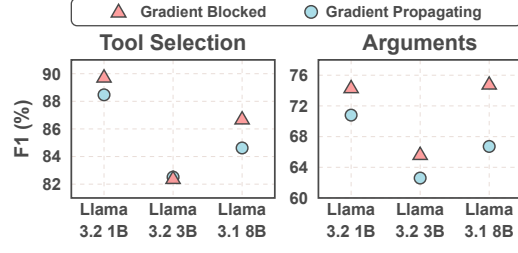


Figure 5: Ablation study on gradient flow.

3.4 Ablation Studies

We perform several ablation studies to test the effectiveness of different modules and hyperparameters. All experiments are conducted on the Llama 3.2-1B.

The Effectiveness of Different Modules. Table 3 and Table 6 (in Appendix E.3) present a step-by-step ablation of `<EOC>` and TCRA to evaluate their contributions to compositional recall. Adding `<EOC>` improves tool selection by 3.4/1.7 and arguments generation by 4.5/2.6 on the 1B/8B backbones, showing that explicit transition termination helps identify procedure boundaries. TCRA further improves tool selection by up to 5.1 and arguments generation by up to 4.2, demonstrating its advantage in refining transition states into accurate next-procedure routing.

Ablation Study on Hyper-Parameters. Figure 3 shows the ablation results for the logit scale and the transition-conditioned routing loss. For the logit scale, the sequence $0.1 \rightarrow 0.5 \rightarrow 1.0$ leads to steady gains in both Tool Selection F1 and Argument Generation F1, while the scale of 2.0 breaks this trend. This result suggests that a small scale gives weak correction, whereas a large scale reduces the accuracy of argument generation. For the transition-conditioned routing loss weight, both tool selection and argument generation reach their best results at 0.1. This finding suggests that TCRA needs a routing strength that is strong enough to improve memory-token selection but not so strong that it harms normal next-token prediction.

Ablation study on gradient design. Figure 5 shows the results under different gradient designs. For argument generation, blocked gradients consistently improve performance over gradient propagation, with a gain of nearly 8 on the 8B model. Meanwhile, tool selection shows no clear drop in performance. The result suggests that propagating adapter gradients into token embeddings may distract the model from standard autoregressive learning and weaken its ability to complete the task. The larger gain in argument generation than in tool selection further supports this explanation.

4 Conclusion

This paper studies multi-step orchestration of LLMs and identify an implicit transition bottleneck in TokMem-style methods. We propose *TapMem*, a lightweight transition-aware framework that exposes procedure boundaries with ETS and improves next-procedure selection through a TCRA. TapMem adjusts only candidate memory-token logits at procedure-selection positions, preserving ordinary full-vocabulary generation. Experiments on compositional and atomic procedure recall show consistent improvements over TokMem, highlighting transition modeling as a key ingredient for reliable tokenized procedural memory.

References

- [1] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, GP Shrivatsa Bhargav, Maxwell Crouse, Chulaka Gunasekara, et al. Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1131–1139, 2024.
- [2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [7] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [8] Pengfei Du. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.
- [9] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.
- [10] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, December 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.emnlp-main.674>.
- [11] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11143–11156, 2024.
- [12] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [13] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. Legomem: Modular procedural memory for multi-agent llm systems for workflow automation. *arXiv preprint arXiv:2510.04851*, 2025.
- [14] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890, 2024.
- [15] N. Hounsby, A. Giurui, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799, 2019. URL <https://proceedings.mlr.press/v97/hounsby19a.html>.
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

- [17] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [18] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- [19] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [20] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021. URL <https://aclanthology.org/2021.emnlp-main.243/>.
- [21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [22] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 4582–4597, 2021. URL <https://aclanthology.org/2021.acl-long.353/>.
- [23] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.
- [24] X. Liu, K. Ji, Y. Fu, W. L. Tam, Z. Du, Z. Yang, and J. Tang. P-Tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 61–68, 2022. URL <https://aclanthology.org/2022.acl-short.8/>.
- [25] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv: 2308.03688*, 2023.
- [26] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh R. N., Liangwei Yang, Silvio Savarese, Juan Carlos Nieves, Huan Wang, Shelby Heinecke, and Caiming Xiong. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/61cce86d180b1184949e58939c4f983d-Abstract-Datasets_and_Benchmarks_Track.html.
- [27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [28] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025.
- [29] Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*, 2023.
- [30] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [31] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://arxiv.org/abs/2305.15334>.
- [32] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

- [33] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- [34] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3391–3409, 2025.
- [35] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [36] Llama Team. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [37] Qwen Team. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [38] R. Wang, X. Han, L. Ji, S. Wang, T. Baldwin, and H. Li. ToolGen: Unified tool retrieval and calling via generation. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=XLMAmowdY>.
- [39] Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Gary Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krma Doshi, Kuntal Kumar Pal, Maitreya Patel, Mehrad Moradshahi, Mihir Parmar, Mirali Purohit, Neeraj Varshney, Phani Rohitha Kaza, Pulkit Verma, Ravsehaj Singh Puri, Rushang Karia, Savan Doshi, Shailaja Keyur Sampat, Siddhartha Mishra, Sujan Reddy A, Sumanta Patro, Tanay Dixit, and Xudong Shen. Supernaturalinstructions: Generalization via declarative instructions on 1600+ NLP tasks. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 5085–5109. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.EMNLP-MAIN.340. URL <https://doi.org/10.18653/v1/2022.emnlp-main.340>.
- [40] Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, et al. Memoryllm: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*, 2024.
- [41] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [42] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- [43] Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to ai memory: A survey on memory mechanisms in the era of llms. *arXiv preprint arXiv:2504.15965*, 2025.
- [44] Z. Wu, Y. Hao, and L. Mou. TokMem: One-token procedural memory for large language models. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RWjEf9PdiJ>.
- [45] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [46] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=wE_vluYUL-X.
- [47] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. Easytool: Enhancing llm-based agents with concise tool instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 951–972, 2025.
- [48] Guibin Zhang, Muxin Fu, and Shuicheng Yan. Memgen: Weaving generative latent memory for self-evolving agents. *arXiv preprint arXiv:2509.24704*, 2025.

- 452 [49] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan
453 Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous
454 agents. *arXiv preprint arXiv:2307.13854*, 2023.
- 455 [50] Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei
456 Lu, Xiaogang Wang, et al. Ghost in the minecraft: Generally capable agents for open-world environments
457 via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*,
458 2023.

460 Contents

461	A. Related Work	15
462	A.1. Memory-Augmented Methods for LLMs	15
463	A.2. Tokenized Procedural Memory	15
464	B. Additional Examples for TapMem	15
465	C. Pseudocode	15
466	D. Additional Experimental Setting	15
467	D.1. Details of Competitors	15
468	D.2. Compositional Recall	17
469	D.3. Atomic Recall	18
470	D.4. Compositional Recall with More Tool Calls	18
471	D.5. Compositional Recall with LM Head Replaced	18
472	E. Additional Experimental Results	18
473	E.1. Expanded Version of Compositional Recall Results	18
474	E.2. Expanded Version of Atomic Recall Results	19
475	E.3. Expanded Version of Ablation Study	19
476	E.4. Compositional Recall with LM Head Replaced.	19
477	F. Comparison of Parameter Count	19
478	G. Limitations and Future Work	20
479	G.1. Limitations	20
480	G.2. Broad Impact and Future Work	20

A Related Work

A.1 Memory-Augmented Methods for LLMs

Memory augmentation allows LLMs to use task-relevant knowledge beyond the immediate input [43, 8]. In in-context learning, demonstrations, instructions, and reasoning traces can serve as temporary textual memory in the prompt [4, 42, 46, 47]. In retrieval-augmented generation, an external memory store provides relevant documents, examples, or tool descriptions at inference time [18, 21, 3, 31, 33, 12, 29]. Recent work on agent-based memory systems further extends text-based memory methods [45, 6, 30, 50, 41, 48]. Some studies also place reusable behavior in trainable parameters or continuous representations, including adapters, soft prompts, prefix tuning, P-Tuning, and LoRA [15, 22, 20, 24, 16, 40]. Although these designs provide useful forms of memory, they still have limitations for procedural reuse. Textual and retrieval-based memory requires extra context construction at each use [43], while behavior stored in parameters is hard to inspect and edit, and it lacks a clear discrete call interface. More importantly, it remains unclear how these memory mechanisms should guide an LLM from one reusable procedure to another within a single autoregressive generation, leaving multi-step procedural orchestration under-specified [9].

A.2 Tokenized Procedural Memory

Tokenized procedural memory [44, 38] provides a more compact formulation, where each reusable procedure is represented as a discrete trainable memory token. A representative example is TokMem [44], which compiles a procedure into a single memory token. The corresponding behavior is then called through standard language-model generation. Long procedure descriptions no longer need to be repeatedly added in the context, and procedure recall remains compatible with autoregressive decoding. However, TokMem still has a key limitation in compositional settings. When multiple procedures needed in sequence [17, 35], the model has to decide when the current memory-guided span should end and which procedure should be selected next. In TokMem, the transition between procedures is left to standard next-token prediction, so procedure termination and next-procedure selection remain implicit and coupled. This makes procedural chaining less stable, especially when the number of procedure calls increases. TapMem is motivated by this missing transition mechanism: it keeps the memory-token form, while using boundary-aware termination and next-procedure routing to make procedure transitions explicit.

B Additional Examples for TapMem

In Figure 6, we present a real tool-use example. TokMem selects an incorrect procedure `<binary_search>` and terminates generation prematurely, whereas TapMem selects the correct procedure and generates all arguments accurately. The example shows that introducing `<EOC>` and routing correction can explicitly strengthen transition modeling, thereby improving the accuracy of procedure selection and argument generation.

C Pseudocode

For clarity, we provide the pseudocode of our *Transition-Aware Procedural Memory* in Algorithm 1 and Algorithm 2.

D Additional Experimental Setting

All experimental results are averaged over three runs.

D.1 Details of Competitors

Here, we provide a detailed summary of the representative baselines spanning textualized context engineering, retrieval-augmented memory, parameter-efficient fine-tuning, and tokenized procedural memory methods.

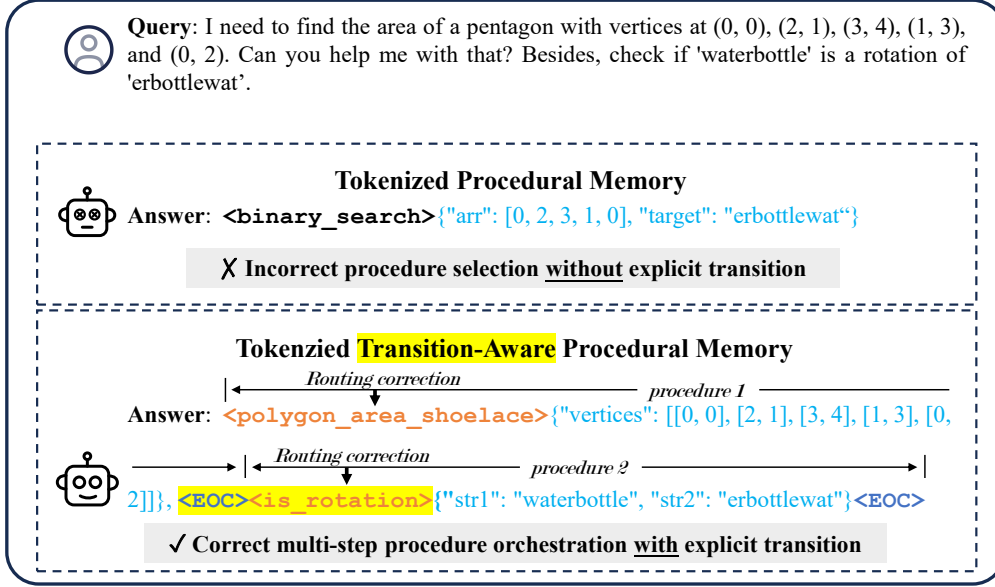


Figure 6: A real example of compositional procedure recall

Algorithm 1 Training

Input: Frozen LLM f_θ ; data $\mathcal{D}$; memory tokens $\mathcal{V}_M$; `<EOC>`; adapter g_ϕ ; scale α ; weight γ .
Output: Trained $\mathbf{E}$, $\mathbf{e}_{\text{eoc}}$, and g_ϕ .

- 1: Initialize $\mathbf{E}$, $\mathbf{e}_{\text{eoc}}$, g_ϕ ; freeze θ .
- 2: **for** each $(\mathbf{q}, \{m^{(j)}, \mathbf{a}^{(j)}\}_{j=1}^J) \in \mathcal{D}$ **do**
- 3: **▷ Build transition-aware supervision**
- 4: $\mathbf{y}' \leftarrow [\mathbf{q}, m^{(1)}, \mathbf{a}^{(1)}, \text{<EOC>}, \dots, m^{(J)}, \mathbf{a}^{(J)}, \text{<EOC>}]$
- 5: $\mathcal{B} \leftarrow \{|\mathbf{q}| + 1\} \cup \{b \mid y'_{b-1} = \text{<EOC>}, y'_b \in \mathcal{V}_M\}$ # procedure-selection boundaries
- 6: **▷ Teacher-forced forward and boundary correction**
- 7: Run f_θ on $\mathbf{y}'$ to get $\{h_r, \ell_r^{\text{LM}}\}$.
- 8: **for** each generated position $r > |\mathbf{q}|$ **do**
- 9: $\tilde{\ell}_r \leftarrow \ell_r^{\text{LM}}$
- 10: **if** $r \in \mathcal{B}$ **then**
- 11: $\pi_r \leftarrow \text{softmax}(g_\phi(\text{stopgrad}(h_r)))$
- 12: $\tilde{\ell}_r[\mathcal{V}_M] \leftarrow \ell_r^{\text{LM}}[\mathcal{V}_M] + \alpha(\log \pi_r + \log K)$ # modify memory logits only
- 13: **end if**
- 14: **end for**
- 15: **▷ Optimize TapMem**
- 16: $\mathcal{L}_{\text{AR}} \leftarrow -\sum_{r>|\mathbf{q}|} \log \text{softmax}(\tilde{\ell}_r)_{y'_r}$
- 17: $\mathcal{L}_{\text{route}} \leftarrow -\frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \log \pi_b(y'_b)$
- 18: Update $\mathbf{E}$, $\mathbf{e}_{\text{eoc}}$, ϕ using $\mathcal{L}_{\text{AR}} + \gamma \mathcal{L}_{\text{route}}$
- 19: **end for**
- 20: **return** $\mathbf{E}$, $\mathbf{e}_{\text{eoc}}$, g_ϕ

- 525 • **Textualized context engineering.** In the atomic setting, the *Base* baseline directly provides the
526 task instruction in text and asks the model to generate the answer without any trainable procedural
527 memory [44]. In the compositional setting, the *ICL* baseline places the available tool descriptions in
528 the prompt and lets the model produce the sequence of function-call arguments directly [4, 31, 33].
- 529 • **Retrieval-augmented textual memory.** In the compositional setting, *RAG* indexes the benchmark-
530 provided tool descriptions and retrieves a small set of candidate tools, e.g., top- k tools, which
531 are then placed in the prompt for generation [21, 18, 31, 33]. In the atomic setting, *RAG* builds a

Algorithm 2 Inference

Input: Frozen LLM f_θ ; trained $\mathbf{E}$, $\mathbf{e}_{\text{eoc}}$, g_ϕ ; query $\mathbf{q}$; memory tokens $\mathcal{V}_M$; scale α ; max length T .
Output: Generated sequence $\hat{\mathbf{y}}$.

```
1:  $\hat{\mathbf{y}} \leftarrow \mathbf{q}$ 
2: for  $r = |\mathbf{q}| + 1$  to  $T$  do
3:    $\triangleright$  Decode with boundary-local routing
4:   Run  $f_\theta$  on  $\hat{\mathbf{y}}$  to get  $h_r, \ell_r^{\text{LM}}$ .
5:    $\tilde{\ell}_r \leftarrow \ell_r^{\text{LM}}$ 
6:   if  $r = |\mathbf{q}| + 1$  or  $\hat{y}_{r-1} = \langle \text{EOC} \rangle$  then
7:      $\pi_r \leftarrow \text{softmax}(g_\phi(h_r))$ 
8:      $\tilde{\ell}_r[\mathcal{V}_M] \leftarrow \ell_r^{\text{LM}}[\mathcal{V}_M] + \alpha(\log \pi_r + \log K)$  # route next procedure
9:   end if
10:  Decode  $\hat{y}_r$  from  $\text{softmax}(\tilde{\ell}_r)$ ; append  $\hat{y}_r$  to  $\hat{\mathbf{y}}$ 
11:  if  $\hat{y}_r = \langle \text{eos} \rangle$  then
12:    return  $\hat{\mathbf{y}}$ 
13:  end if
14: end for
15: return  $\hat{\mathbf{y}}$ 
```

532 non-parametric retrieval index from Natural Instructions task demonstrations and retrieves the most
533 similar demonstration according to the input query [21, 18, 39].

- 534 • **Parameter-efficient fine-tuning.** We include *LoRA* as a parameter-efficient learning baseline for
535 compositional tool invocation. LoRA adapts the frozen backbone through a small set of low-rank
536 update matrices inserted into selected attention projections [16]. This baseline evaluates whether
537 reusable procedures can be captured by lightweight parametric adaptation alone.
- 538 • **TokMem.** We use *TokMem* as the baseline of tokenized procedural memory. It represents each
539 procedure as a trainable token and invokes procedures through autoregressive token generation [44].

540 D.2 Compositional Recall

541 **Datasets** For compositional recall, we use a multi-tool calling benchmark synthesized from API-
542 Gen [26]. APIGen is an automated data generation pipeline for constructing verifiable and diverse
543 function-calling datasets. It collects 3,673 executable APIs across 21 categories and uses them to
544 synthesize natural-language query-function-call pairs in a scalable and structured way. The released
545 APIGen dataset contains 60,000 high-quality entries and covers multiple query styles, including
546 simple, multiple, parallel, and parallel-multiple function-calling cases, making it a suitable resource
547 for evaluating and training LLMs on tool selection and argument generation.

548 **Data Construction** Following TokMem [44], we remove web-search-related tools from the dataset
549 and select the 100 tools with the most samples. We use the first 50 tools to construct the training
550 dataset for the adaptation stage and the remaining 50 tools to construct the dataset for the main training
551 stage of TapMem. For TapMem with adaptation, the training data used in the second stage is the same
552 as the training data used by TapMem without adaptation. Each tool contains 50 query-response pairs,
553 which are treated as atomic samples and included as part of the training data. We further use these
554 data to synthesize compositional data by randomly sampling atomic samples and combining them
555 into compositional samples, where conjunctions are inserted into the queries. Each compositional
556 sample contains at least 2 calls and at most 4 calls. All samples in the test set are compositional
557 samples. In total, 2,500 atomic samples and 2,500 compositional samples are included in the training
558 set, while 500 samples are included in the test set. Tools used in adaptation phase are listed as Table 9.
559 Tools used for our method are listed as Table 10.

560 **Metrics** For compositional function-calling, we employ two primary metrics: Tool Selection F1,
561 $\frac{2P_{\text{tool}}R_{\text{tool}}}{P_{\text{tool}}+R_{\text{tool}}}$, and Arguments Generation F1, $\frac{2P_{\text{arg}}R_{\text{arg}}}{P_{\text{arg}}+R_{\text{arg}}}$. Here, P_{tool} and R_{tool} are computed from
562 the overlap between the gold and predicted procedural memory token sets $\mathcal{T} = \{m^{(j)}\}_{j=1}^J$ and

563 $\hat{T} = \{\hat{m}^{(j)}\}_{j=1}^J$, while P_{arg} and R_{arg} are computed from the overlap between the gold and predicted
 564 full function-call sets $\mathcal{C} = \{(m^{(j)}, \mathbf{a}^{(j)})\}_{j=1}^J$ and $\hat{\mathcal{C}} = \{(\hat{m}^{(j)}, \hat{\mathbf{a}}^{(j)})\}_{j=1}^J$.

565 **Transition Error** We evaluate transition errors at both the token level and the sample level. At the
 566 token level, each gold transition step is considered separately. The first procedure in an example is
 567 treated as the initial decision, whereas each subsequent procedure is treated as a transition decision
 568 because it requires the model to move from one procedure to the next. A token-level transition error
 569 occurs when the model predicts an incorrect procedure at any subsequent procedure-selection step.
 570 If the model stops early and misses a later gold procedure, that step is also counted as incorrect.
 571 Extra predicted procedures beyond the gold sequence are ignored under this metric. At the sample
 572 level, we examine whether an example contains any transition error. If any subsequent procedure-
 573 selection step is predicted incorrectly, the whole example is counted as having a transition error.
 574 Therefore, the sample-level metric measures how often a multi-procedure example contains at least
 575 one transition failure, whereas the token-level metric measures how many individual transition
 576 decisions are incorrect.

577 D.3 Atomic Recall

578 **Datasets** Super-Natural Instructions (SNI) [39] is a large-scale collection of instruction-formatted
 579 NLP tasks. Each task consists of a natural-language definition, positive and negative examples, and
 580 multiple input-output instances. The dataset is commonly used to evaluate instruction following
 581 and cross-task generalization. It covers 1,616 tasks across 76 task types, where each task is treated
 582 as a reusable procedure and represented by a procedural memory token. Each input is associated
 583 with exactly one target procedure, so the model only needs to select a single memory token before
 584 generating the corresponding answer.

585 **Metrics** For atomic memory recall, we use procedural memory token routing accuracy to measure
 586 the correctness of memory-token selection and ROUGE-L to evaluate the quality of the generated
 587 textual response.

588 **Data Construction** To construct the dataset used in our experiments, we first remove cross-lingual
 589 tasks from the SNI dataset and then select tasks with more than 560 samples. From the remaining
 590 tasks, we sample 700 tasks, with 80 training samples, 10 validation samples, and 20 test samples
 591 drawn from each task. Tasks used in our experiment are shown in Figure 9.

592 D.4 Compositional Recall with More Tool Calls

593 For compositional recall with more tool calls, we further extend the synthetic dataset by using the
 594 same atomic samples to construct longer samples. Each sample contains up to 10 calls. The training
 595 set includes 2,500 atomic samples and 5,500 compositional samples. Among the compositional
 596 samples, the proportions of samples with different numbers of tool calls are kept consistent.

597 D.5 Compositional Recall with LM Head Replaced

598 We conduct a replace-head experiment to test whether the adapter should directly replace the LM
 599 head or only calibrate the logits through an additive correction. In this setting, the adapter directly
 600 replaces the LM logits at boundary positions instead of conducting TCRA.

601 E Additional Experimental Results

602 E.1 Expanded Version of Compositional Recall Results

603 More detailed results about compositional recall with more tool calls are provided in Table 4. Our
 604 method consistently outperforms TokMem across all numbers of tool calls, and the advantage becomes
 605 more evident and stable as the number of tool calls increases.

Table 4: Results of compositional recall with more tool calls on Llama 3.2 1B across 2-to-10 procedure calls. Δ denotes the improvement of Ours over TokMem.

Calls	Method	Tool Selection F1(%)	Δ Tool Selection F1(%)	Arguments F1(%)	Δ Arguments F1(%)
2	TokMem	89.0		75.0	
	Ours	90.7	+1.7	77.6	+2.6
3	TokMem	89.3		76.8	
	Ours	94.4	+5.1	82.3	+5.5
4	TokMem	88.8		72.9	
	Ours	91.1	+2.3	75.1	+2.2
5	TokMem	88.9		75.4	
	Ours	92.2	+3.3	78.2	+2.8
6	TokMem	91.0		76.9	
	Ours	92.1	+1.1	78.5	+1.6
7	TokMem	87.5		70.9	
	Ours	91.9	+4.4	75.3	+4.4
8	TokMem	88.5		72.8	
	Ours	91.8	+3.3	76.9	+4.1
9	TokMem	86.8		71.5	
	Ours	92.7	+5.9	77.3	+5.8
10	TokMem	83.8		67.9	
	Ours	89.1	+5.3	71.6	+3.7
Avg.	TokMem	88.5		73.8	
	Ours	92.0	+3.5	77.4	+3.6

E.2 Expanded Version of Atomic Recall Results

Table 5 compares TokMem with our method on the 30 categories with the largest number of samples. Although performance slightly decreases on a subset of tasks, it improves on most tasks, and the gains are relatively clear. Performance also improves on the three tasks with the largest number of samples.

E.3 Expanded Version of Ablation Study

Table 6 presents the detailed ablation results of different modules in our method on three models. The results show that the <EOC> module outperforms the unmodified TokMem baseline in most call-count settings. TCRA further improves the performance and achieves the best results across all call-count settings.

E.4 Compositional Recall with LM Head Replaced

The results are reported in Table 7. It shows that direct replacement is less stable. On Llama 1B, replace-head achieves similar Tool Selection F1 to ours (89.9 vs. 89.7), showing that the adapter can learn useful routing signals, but its Arguments F1 is slightly lower (73.9 vs. 74.3). On 3B, the degradation is clearer, with Tool Selection F1 dropping from 82.4 to 80.3 and Arguments F1 from 65.5 to 61.0. This suggests that the original logits already contain useful calibration from the query, previous calls, <EOC> boundary, and memory-token embeddings. Replacing them discards this calibration, whereas TCRA preserves it and only corrects the fragile routing decision.

F Comparison of Parameter Count

In Table 8, we report the number of trainable parameters for the parameterized methods used in this paper across models of different scales. Our method requires substantially fewer trainable parameters

627 than fine-tuning. Although it has more trainable parameters than TokMem, it still follows a linear
628 growth trend with the embedding dimension.

629 **G Limitations and Future Work**

630 **G.1 Limitations**

631 Our method focuses on multi-step procedural memory orchestration. Although it shows promising
632 performance, several limitations remain. First, our current study assumes a relatively fixed procedural
633 memory setting and does not explicitly address how procedural memories can be updated over time.
634 Second, due to computational constraints, we have not yet evaluated our method on larger-scale
635 models. Finally, following related work, we conduct experiments only on decoder-only architectures,
636 and the applicability of our method to other model architectures remains unexplored.

637 **G.2 Broad Impact and Future Work**

638 TapMem may have broader impact by improving the reliability of procedural memory orchestration
639 in multi-step language-model systems. More accurate procedure transitions can benefit tool use,
640 task automation, and other applications that require models to follow reusable procedures across
641 multiple steps. To further extend this potential, several important directions can be explored, including
642 efficient mechanisms for updating procedural memories, evaluation on larger backbone models, and
643 extension beyond decoder-only architectures. In particular, more flexible memory-update mechanisms
644 would allow procedural memories to adapt to newly introduced or revised procedures with lower
645 training cost. Evaluating TapMem under broader model scales and architectures would also help
646 clarify its generality and support its integration into more practical language-model systems. These
647 efforts would make procedural memory systems more adaptive, scalable, and broadly applicable in
648 real-world settings.

task1 quoref question generation
task2 quoref answer generation
task18 mctaco temporal reasoning presence
task23 cosmosqa question generation
task24 cosmosqa answer generation
task25 cosmosqa incorrect answer generation
task27 drop answer type generation
task28 drop answer generation
task29 winogrande full object
task30 winogrande full person
task31 winogrande question generation object
task32 winogrande question generation person
task33 winogrande answer generation
task34 winogrande question modification object
task35 winogrande question modification person
task36 qasc topic word to generate related fact
task37 qasc generate related fact
task38 qasc combined fact
task39 qasc find overlapping words
task40 qasc question generation
task41 qasc answer generation
task42 qasc incorrect option generation
task43 essential terms answering incomplete questions
task44 essential terms identifying essential words
task48 multirc question generation
task50 multirc answerability
task51 multirc correct answer single sentence
task55 multirc write incorrect answer
task57 multirc classify incorrect answer
task58 multirc question answering
task60 ropes question generation
task61 ropes answer generation
task63 first i elements
task64 all elements except first i
task65 timetravel consistent sentence classification
task66 timetravel binary consistency classification
task67 abductivenli answer generation
task68 abductivenli incorrect answer generation
task69 abductivenli classification
task71 abductivenli answer generation
task72 abductivenli answer generation
task73 commonsenseqa answer generation
task74 squad1.1 question generation
task75 squad1.1 answer generation
task76 splash correcting sql mistake
task77 splash explanation to sql
task78 all elements except last i
task79 conala concat strings
task80 piqa answer generation
task81 piqa wrong answer generation
task82 babi t1 single supporting fact question generation
task83 babi t1 single supporting fact answer generation
task84 babi t1 single supporting fact identify relevant fact
task85 unnatural addsub arithmetic
task87 new operator addsub arithmetic
task88 identify typo verification
task89 swap words verification
task90 equation learner algebra
task91 all elements from index i to j
task92 check prime classification
task93 conala normalize lists
task94 conala calculate mean
task95 conala max absolute value
task96 conala list index subtraction
task97 conala remove duplicates
task98 conala list intersection
task99 reverse elements between index i and j
task100 concatenate all elements from index i to j
task101 reverse and concatenate all elements from index i to j
task102 commongen sentence generation
task103 facts2story long text generation
task104 semeval 2019 task10 closed vocabulary mathematical answer generation
task105 story cloze-roctories sentence generation
task106 scruples ethical judgment
task108 contextualabusedetection classification
task109 smsspamcollection spamsmsdetection
task110 logic2text sentence generation
task111 asset sentence simplification
task112 asset simple sentence identification
task113 count frequency of letter
task114 is the given word longest
task116 com2sense commonsense reasoning
task123 conala sort dictionary
task124 conala pair averages
task125 conala pair differences
task126 scan structured text generation command action all
task127 scan long text generation action command all
task128 scan structured text generation command action short
task129 scan long text generation action command short
task130 scan structured text generation command action long
task131 scan long text generation action command long
task132 dais text modification
task133 winowhy reason plausibility detection
task141 odd-man-out classification category
task142 odd-man-out classification no category
task143 odd-man-out classification generate category
task144 subjq question answering
task145 afs argument similarity death penalty
task146 afs argument similarity gun control
task147 afs argument similarity gay marriage
task148 afs argument quality gay marriage
task149 afs argument quality death penalty
task150 afs argument quality gun control
task151 tomqa find location easy clean
task152 tomqa find location easy noise
task153 tomqa find location hard clean
task154 tomqa find location hard noise
task156 codah classification adversarial
task157 count vowels and consonants
task158 count frequency of words
task159 check frequency of words in sentence pair
task160 replace letter in a sentence
task161 count words containing letter
task162 count words starting with letter
task163 count words ending with letter
task164 mscscript question answering text
task165 mscscript question answering commonsense
task167 strategyqa question generation
task168 strategyqa question decomposition
task169 strategyqa sentence generation
task176 break decompose questions
task177 para-nmt paraphrasing
task178 quartz question answering
task182 duorc question generation
task183 rhyme generation
task184 break generate question
task184 snli entailment to neutral text modification
task185 snli contradiction to neutral text modification
task186 snli contradiction to entailment text modification
task187 snli entailment to contradiction text modification
task188 snli neutral to entailment text modification
task189 snli neutral to contradiction text modification
task190 snli classification
task193 duorc question generation
task194 duorc answer generation
task196 sentiment140 answer generation
task197 mnli domain answer generation
task198 mnli domain classification
task199 mnli classification
task200 mnli entailment classification
task201 mnli neutral classification
task202 mnli contradiction classification
task203 mnli sentence generation
task204 mnli same genre classification
task205 remove even elements
task206 collatz conjecture
task207 max element lists
task208 combinations of list
task209 stancedetection classification
task210 logic2text structured text generation
task211 logic2text classification
task213 roctories correct ending classification
task215 roctories incorrect answer generation
task216 roctories correct answer generation
task217 roctories ordering answer generation
task218 roctories swap order answer generation
task219 roctories title answer generation
task220 roctories title classification
task221 roctories two choice classification
task222 roctories two choice slotting classification
task223 quartz explanation generation
task224 scruples anecdotes ethical judgment
task227 clarif classification
task228 arc answer generation easy
task229 arc answer generation hard
task230 iirc passage classification
task231 iirc link classification
task232 iirc link number classification
task233 iirc link exists classification
task234 iirc passage line answer generation
task235 iirc question from subtext answer generation
task236 iirc question from passage answer generation
task237 iirc answer from subtext answer generation
task238 iirc answer from passage answer generation
task239 tweetqa answer generation
task240 tweetqa question generation
task241 tweetqa classification
task242 tweetqa classification
task243 count elements in set intersection
task244 count elements in set union
task245 check presence in set intersection
task246 dream question generation
task247 dream answer generation
task248 dream classification
task249 enhanced wsc pronoun disambiguation
task265 paper reviews language identification
task267 concatenate and reverse all elements from index i to j
task268 casehold legal answer generation
task269 csrg counterfactual story generation
task270 csrg counterfactual context generation
task273 europarl classification
task275 enhanced wsc paraphrase generation
task276 enhanced wsc classification
task287 stereoset sentence generation antistereotype
task279 stereoset classification stereotype
task280 stereoset classification stereotype type
task281 points of correspondence
task282 scruples event time
task283 dream incorrect answer generation
task284 imdb classification
task285 imdb answer generation
task286 olid offense judgment
task287 casehold legal incorrect answer generation
task288 gigaword summarization
task289 gigaword summarization
task290 tellmewhy question answerability
task291 semeval 2020 task4 commonsense validation
task292 storycommonsense character text generation
task293 storycommonsense emotion text generation
task294 storycommonsense motiv text generation
task295 semeval 2020 task4 commonsense reasoning
task296 storycloze correct end classification
task297 storycloze incorrect end classification
task298 storycloze correct end classification
task299 storycloze sentence generation
task300 storycloze order generation
task301 record question generation
task302 record classification
task303 record incorrect answer generation
task305 jeopardy answer generation normal
task306 jeopardy answer generation double
task307 jeopardy answer generation final
task308 jeopardy answer generation all
task309 race answer generation
task310 race classification
task311 race question generation
task314 europarl sv-en classification
task315 europarl sv-en language identification
task316 crows-pairs classification stereotype
task318 stereoset classification gender
task319 stereoset classification profession
task320 stereoset classification race
task322 jigsaw classification threat
task323 jigsaw classification sexually explicit

Figure 7: Part 1 of the tasks used in the atomic recall experiment.

task325 jigsaw classification identity attack
task326 jigsaw classification obscene
task327 jigsaw classification toxic
task328 jigsaw classification insult
task329 gap classification
task330 gap answer generation
task331 gap incorrect answer generation
task332 tellmewhy answer generation
task333 hateeval classification hate en
task335 hateeval classification aggressive en
task337 hateeval classification individual en
task339 record answer generation
task340 winomt classification gender pro
task341 winomt classification gender anti
task342 winomt classification profession pro
task343 winomt classification profession anti
task344 hybridqa answer generation
task345 hybridqa answer generation
task346 hybridqa classification
task347 hybridqa incorrect answer generation
task348 squad2.0 unanswerable question generation
task349 squad2.0 answerable unanswerable question classification
task350 winomt classification gender identifiability pro
task351 winomt classification gender identifiability anti
task353 casino classification negotiation elicit pref
task355 casino classification negotiation other need
task356 casino classification negotiation self need
task357 casino classification negotiation small talk
task359 casino classification negotiation vouch fair
task360 spolin yesand response generation
task361 spolin yesand prompt response classification
task362 spolin yesand prompt response sub classification
task363 sst2 polarity classification
task365 synthetic remove vowels
task366 synthetic return primes
task367 synthetic remove floats
task368 synthetic even or odd calculation
task369 synthetic remove odds
task370 synthetic remove divisible by 3
task371 synthetic product of list
task372 synthetic palindrome numbers
task374 synthetic pos or neg calculation
task376 reverse order of words
task377 remove words of given length
task378 reverse words of given length
task379 agnews topic classification
task380 boolq yes no question
task381 boolq question generation
task382 hybridqa answer generation
task383 matres classification
task384 socialqa question classification
task385 socialqa incorrect answer generation
task386 semeval 2018 task3 irony detection
task387 semeval 2018 task3 irony classification
task388 torque token classification
task389 torque generate temporal question
task390 torque text span selection
task391 causal relationship
task392 inverse causal relationship
task397 semeval 2018 task1 tweet anger detection
task398 semeval 2018 task1 tweet joy detection
task399 semeval 2018 task1 tweet sadness detection
task400 paws paraphrase classification
task401 numeric fused head reference
task402 grailqa paraphrase generation
task403 creak commonsense inference
task404 grailqa paraphrase validation
task405 narrativeqa question generation
task413 mickey en sentence perturbation generation
task418 persent title generation
task419 persent answer generation
task420 persent document sentiment classification
task422 persent sentence sentiment verification
task423 persent document sentiment verification
task426 hindienglish corpora hi-en classification
task427 hindienglish corpora hi-en language identification
task428 senteval inversion
task429 senteval tense
task430 senteval subject count
task431 senteval object count
task437 alt en ja answer generation
task440 eng guj parallel corpus gu-en classification
task441 eng guj parallel corpus gu-en language identification
task442 com qa paraphrase question generation
task443 com qa ans question generation
task444 com qa question paraphrases answer generation
task453 swag answer generation
task454 swag incorrect answer generation
task455 swag context generation
task456 matres intention classification
task457 matres conditional classification
task458 matres negation classification
task460 qasper answer generation
task461 qasper question generation
task469 mrqa answer generation
task470 mrqa question generation
task471 haspart answer generation
task472 haspart classification
task475 yelp polarity classification
task476 cls english books classification
task477 cls english dvd classification
task478 cls english music classification
task488 extract all alphabetical elements from list in order
task493 review polarity classification
task494 review polarity answer generation
task495 semeval headline classification
task496 semeval answer generation
task497 extract all numbers from list in order
task498 scruples anecdotes whoiswrong classification
task499 extract and add all numbers from list
task501 scruples anecdotes post type verification
task502 scruples anecdotes whoiswrong verification
task503 scruples anecdotes isanswerable
task504 count all alphabetical elements in list
task505 count all numerical elements in list
task506 position of all alphabetical elements in list
task507 position of all numerical elements in list
task508 scruples dilemmas more ethical isidentifiable
task509 collate of all alphabetical and numerical elements in list separately
task510 reddit tfidf title summarization
task511 reddit tfidf long text summarization
task516 senteval conjoints inversion
task517 emo classify emotion of dialogue
task518 emo different dialogue emotions
task519 aquamuse question generation
task520 aquamuse answer given in passage
task521 trivia question classification
task523 find if numbers or alphabets are more in list
task532 europarl en-es classification
task533 europarl es-en language identification
task547 alt translation entk en
task550 discofuse sentence generation
task563 discofuse answer generation
task565 circa answer generation
task566 circa classification
task567 circa text generation
task568 circa question generation
task569 recipe nlg text generation
task570 recipe nlg ner generation
task571 recipe nlg ner generation
task572 recipe nlg text generation
task573 air dialogue classification
task574 air dialogue sentence generation
task575 air dialogue classification
task576 curiosity dialogs answer generation
task577 curiosity dialogs classification
task578 curiosity dialogs answer generation
task580 socialqa answer generation
task581 socialqa question generation
task582 naturalquestion answer generation
task583 udeps eng coarse pos tagging
task584 udeps eng fine pos tagging
task585 preposition classification
task586 amazonfood polarity classification
task587 amazonfood polarity correction classification
task588 amazonfood rating classification
task590 amazonfood summary correction classification
task591 sciq answer generation
task592 sciq incorrect answer generation
task593 sciq explanation generation
task594 sciq question generation
task595 mocha answer generation
task596 mocha question generation
task600 find the longest common substring in two strings
task605 find the longest common subsequence in two lists
task606 sum of all numbers in list between positions i and j
task608 sbic sexual offense binary classification
task609 sbic potentially offense binary classification
task610 conllpp ner
task611 mutual multi turn dialogue
task613 politfact text generation
task614 glucose cause event detection
task615 moviesqa answer generation
task616 cola classification
task617 amazonreview category text generation
task618 amazonreview summary text generation
task620 ohsumed medical subject headings answer generation
task621 ohsumed yes no numerical answer generation
task622 replace alphabets in a list by their position in english alphabet
task624 ohsumed question answering
task629 dbpedia 14 classification
task630 dbpedia 14 classification
task631 dbpedia 14 incorrect answer generation
task632 dbpedia 14 classification
task633 dbpedia 14 answer generation
task634 allegro reviews classification
task635 allegro reviews answer generation
task636 extract and sort unique alphabets in a list
task637 extract and sort unique digits in a list
task638 multi woz classification
task645 summarization
task649 race blank question generation
task668 extreme abstract summarization
task669 ambigqa answer generation
task670 ambigqa question generation
task671 ambigqa text generation
task672 nummersense
task673 google wellformed query classification
task674 google wellformed query sentence generation
task675 google wellformed query sentence generation
task676 ollie relationship answer generation
task677 ollie sentence answer generation
task679 hope edi english text classification
task681 hope edi malayalam text classification
task682 online privacy policy text classification
task683 online privacy policy text purpose answer generation
task684 online privacy policy text information type generation
task738 perspective classification
task743 eurlx summarization
task744 eurlx classification
task746 yelp restaurant review classification
task747 glucose cause emotion detection
task748 glucose reverse cause event detection
task749 glucose reverse cause emotion detection
task750 aqua multiple choice answering
task755 find longest substring and replace its sorted lowercase version in both lists
task756 find longest substring and return all unique alphabets in it
task767 craigslist bargains classification
task769 qed summarization
task820 protoqa answer generation
task821 protoqa question generation
task823 peixian-rtgender sentiment analysis
task827 copa commonsense reasoning
task831 piga fren classification
task832 poleval2019 mt classification
task834 mathdataset classification
task843 financial phrasebank classification
task844 financial phrasebank classification
task845 pubmedqa question generation
task846 pubmedqa classification
task847 pubmedqa question generation
task848 pubmedqa classification
task849 pubmedqa answer generation

Figure 8: Part 2 of the tasks used in the atomic recall experiment.

task850 synthetic longest palindrome
task851 synthetic multiply evens
task852 synthetic multiply odds
task864 asdiv singleop question answering
task865 mawps addsub question answering
task866 mawps multidiv question answering
task868 mawps singleop question answering
task870 msmarco answer generation
task875 emotion classification
task879 schema guided dstc8 classification
task880 schema guided dstc8 classification
task881 schema guided dstc8 classification
task896 miam language classification
task897 freebase qa topic question generation
task898 freebase qa answer generation
task899 freebase qa topic generation
task900 freebase qa category classification
task902 deceptive opinion spam classification
task903 deceptive opinion spam classification
task904 hate speech offensive classification
task923 event2mind classifier
task924 event2mind word generation
task933 wiki auto style transfer
task936 defensible nil snli classification
task937 defensible nil social classification
task955 wiki auto style transfer
task956 leetcode 420 strong password check
task958 e2e nlg text generation parse
task970 sherlic causal relationship
task976 pib indian language identification
task1135 xcsr en commonsense mc classification
task1154 bard analogical reasoning travel
task1156 bard analogical reasoning tools
task1157 bard analogical reasoning rooms for containers
task1159 bard analogical reasoning containers
task1162 coda19 title classification
task1163 coda19 section classification
task1164 coda19 section correction classification
task1168 brown coarse pos tagging
task1186 nne hrngo classification
task1187 politifact classification
task1196 atomic classification oeffect
task1197 atomic classification oreact
task1198 atomic classification owant
task1199 atomic classification xattr
task1200 atomic classification xeffect
task1201 atomic classification xintent
task1202 atomic classification xneed
task1203 atomic classification xreact
task1204 atomic classification hinderedby
task1206 atomic classification isbefore
task1207 atomic classification atlocation
task1208 atomic classification xreason
task1209 atomic classification objectuse
task1210 atomic classification madeupof
task1211 atomic classification hassubevent
task1212 atomic classification hasproperty
task1213 atomic classification desires
task1214 atomic classification xwant
task1215 atomic classification capableof
task1216 atomic classification causes
task1217 atomic answer generation
task1283 hrngo quality classification
task1284 hrngo informativeness classification
task1285 kpa keypoint matching
task1286 openbookqa question answering
task1287 glue qqp paraphrasing
task1288 glue mrpc paraphrasing
task1289 trec classification
task1290 xsum summarization
task1292 yelp review full text categorization
task1293 kill tasks hotpotqa question answering
task1294 wiki qa answer verification
task1295 adversarial qa question answering
task1296 wiki hop question answering
task1297 qasc question answering
task1308 amazonreview category classification
task1309 amazonreview summary classification
task1310 amazonreview rating classification
task1311 amazonreview rating classification
task1312 amazonreview polarity classification
task1313 amazonreview polarity classification
task1325 qa zre question generation on subject relation
task1327 qa zre answer generation from question
task1328 qa zre relation generation from question
task1338 peixian equity evaluation corpus sentiment classifier
task1339 peixian equity evaluation corpus text completion
task1340 msr text compression compression
task1342 amazon us reviews title
task1343 amazon us reviews rating
task1344 glue entailment classification
task1345 glue qqp question paraphrasing
task1346 glue cola grammatical correctness classification
task1347 glue sts-b similarity classification
task1354 sent comp classification
task1355 sent comp summarization
task1356 xisum title generation
task1357 xisum summary generation
task1358 xisum title generation
task1359 numer sense answer generation
task1360 numer sense multiple choice qa generation
task1361 movierationales classification
task1364 hans answer generation
task1368 healthfact sentence generation
task1369 healthfact sentence generation
task1370 newscomm classification
task1376 quarel correct answer generation
task1379 quarel incorrect answer generation
task1380 quarel correct option generation
task1381 quarel incorrect option generation
task1382 quarel write correct answer
task1383 quarel write incorrect answer
task1384 deal or no dialog classification
task1385 anli r1 entailment
task1386 anli r2 entailment
task1387 anli r3 entailment
task1389 hellaswag completion
task1390 wscfixed coreference
task1391 winogrande easy answer generation
task1392 superglue multirc answer verification
task1398 obqa question generation
task1399 obqa answer generation
task1400 obqa incorrect answer generation
task1407 dart question generation
task1408 dart similarity classification
task1409 dart text generation
task1410 dart relationship extraction
task1411 dart subject identification
task1412 web questions question answering
task1413 dart object identification
task1415 youtube caption corrections grammar correction
task1416 youtube caption corrections incorrect grammar classification
task1418 bless semantic relation classification
task1419 mathqa gain
task1420 mathqa general
task1421 mathqa other
task1422 mathqa physics
task1423 mathqa geometry
task1429 evalution semantic relation classification
task1434 head qa classification
task1439 doqa cooking isanswerable
task1443 string to number
task1444 round power of two
task1445 closest integers
task1446 farthest integers
task1447 drug extraction ade
task1481 gene extraction bc2gm dataset
task1495 adverse drug event classification
task1499 dstc3 summarization
task1500 dstc3 classification
task1501 dstc3 answer generation
task1502 hatexplain classification
task1503 hatexplain classification
task1504 hatexplain answer generation
task1505 root09 semantic relation classification
task1508 wordnet antonyms
task1510 evalution relation extraction
task1516 impmpres naturallanguagel inference
task1517 limit classification
task1518 limit answer generation
task1520 qa srl answer generation
task1529 scitail1.1 classification
task1532 daily dialog emotion classification
task1533 daily dialog formal classification
task1534 daily dialog question classification
task1535 daily dialog uniqueness classification
task1536 daily dialog happiness classification
task1540 parsed pdfs summarization
task1541 agnews classification
task1542 every ith element from starting
task1551 every ith element from kth element
task1552 scitail question generation
task1553 cnn dailymail summarization
task1554 scitail classification
task1555 scitail answer generation
task1556 scitail passage generation
task1557 jfleg answer generation
task1558 jfleg incorrect answer generation
task1559 blimp binary classification
task1560 blimp binary classification
task1580 eqasc-perturbed question generation
task1581 eqasc-perturbed answer generation
task1583 bless meronym classification
task1584 evalution meronym classification
task1585 root09 hypernym generation
task1586 scifact title generation
task1587 scifact classification
task1589 scifact classification
task1592 yahoo answers topics classification
task1593 yahoo answers topics classification
task1594 yahoo answers topics question generation
task1595 event2mind text generation 1
task1596 event2mind text generation 2
task1598 nyc long text generation
task1599 smcalfow classification
task1600 smcalfow sentence generation
task1601 webquestions answer generation
task1602 webquestion question generation
task1603 smcalfow sentence generation
task1604 ethos text classification
task1608 xquad en answer generation
task1612 sick label classification
task1613 sick given category generate sentence
task1614 sick text modify
task1615 sick classify b relation a
task1618 cc aligned classify tel eng
task1621 meny020k-mt en yo language identification
task1622 disfl qa text modification
task1623 disfl qa disfluent question classification
task1624 disfl qa question yesno classification
task1625 disfl qa answer generation
task1630 openpi classification
task1631 openpi answer generation
task1640 aqa1.0 answerable unanswerable question classification
task1645 medical question pair dataset text classification
task1659 title generation
task1660 super glue question generation
task1661 super glue classification
task1669 md gender bias text modification
task1670 md gender bias text modification
task1678 mathqa answer selection
task1711 poki text generation
task1712 poki classification
task1714 conval3 sentence generation
task1720 civil comments toxicity classification
task1721 civil comments obscenity classification
task1723 civil comments sexuallyexplicit classification
task1724 civil comments insult classification
task1725 civil comments severtotoxicity classification
task1727 wiqua what is the effect
task1728 web nlg data to text
task1729 personachat generate next
task1731 quartz question answering

Figure 9: Part 3 of the tasks used in the atomic recall experiment.

Table 5: **Fine-grained** atomic recall results for the top 30 categories by task count. Each category is shown as a paired comparison between TokMem and our method; Δ are computed relative to TokMem.

Category	Tasks	Method	Routing Acc	Δ Acc	Rouge-L	Δ Rouge-L
Question Answering	86	TokMem	0.7564		33.2114	
		Ours	0.8083	+0.0519	33.5056	+0.2941
Program Execution	71	TokMem	0.9244		31.2673	
		Ours	0.9437	+0.0192	33.3136	+2.0462
Question Generation	44	TokMem	0.8580		24.4871	
		Ours	0.8284	-0.0295	24.7971	+0.3099
Sentiment Analysis	43	TokMem	0.9709		68.9979	
		Ours	0.9837	+0.0128	66.2467	-2.7512
Text Matching	32	TokMem	0.9313		74.9027	
		Ours	0.9354	+0.0042	74.4146	-0.4882
Text Categorization	30	TokMem	0.9589		49.9105	
		Ours	0.9778	+0.0189	51.0200	+1.1094
Toxic Language Detection	23	TokMem	0.9754		58.2272	
		Ours	0.9877	+0.0123	56.7548	-1.4724
Commonsense Classification	22	TokMem	0.9955		53.0051	
		Ours	0.9992	+0.0038	52.7273	-0.2778
Information Extraction	18	TokMem	0.8343		43.9643	
		Ours	0.8444	+0.0102	43.0107	-0.9536
Textual Entailment	18	TokMem	0.9046		50.1235	
		Ours	0.8972	-0.0074	49.1667	-0.9568
Wrong Candidate Generation	16	TokMem	0.7833		21.9087	
		Ours	0.8271	+0.0438	24.5013	+2.5926
Sentence Composition	14	TokMem	0.8071		36.4334	
		Ours	0.8619	+0.0548	36.9326	+0.4992
Text Completion	14	TokMem	0.9143		25.9161	
		Ours	0.9179	+0.0036	28.0092	+2.0931
Title Generation	14	TokMem	0.7655		26.5221	
		Ours	0.7964	+0.0310	29.0154	+2.4934
Misc.	12	TokMem	0.6389		15.3407	
		Ours	0.6806	+0.0417	15.7341	+0.3934
Question Understanding	12	TokMem	0.9472		43.3202	
		Ours	0.9444	-0.0028	44.3143	+0.9941
Answerability Classification	10	TokMem	0.9850		70.3889	
		Ours	0.9950	+0.0100	72.0278	+1.6389
Dialogue Generation	10	TokMem	0.8850		21.9427	
		Ours	0.8900	+0.0050	22.2786	+0.3359
Language Identification	9	TokMem	0.9204		60.3704	
		Ours	0.9648	+0.0444	67.2963	+6.9259
Summarization	9	TokMem	0.7333		23.3743	
		Ours	0.6815	-0.0519	23.4391	+0.0648
Coreference Resolution	8	TokMem	0.9021		35.1602	
		Ours	0.9750	+0.0729	43.5228	+8.3626
Linguistic Probing	8	TokMem	0.9958		54.5833	
		Ours	0.9917	-0.0042	53.5417	-1.0417
Question Rewriting	8	TokMem	0.7979		57.6151	
		Ours	0.8146	+0.0167	52.4457	-5.1694
Text Quality Evaluation	8	TokMem	0.9771		49.0139	
		Ours	0.9812	+0.0042	50.2001	+1.1862
Text to Code	8	TokMem	0.5979		42.7856	
		Ours	0.5604	-0.0375	42.4521	-0.3335
Data to Text	7	TokMem	0.8262		35.0406	
		Ours	0.8905	+0.0643	33.4534	-1.5873
Fill in The Blank	7	TokMem	0.9476		17.8706	
		Ours	0.9190	-0.0286	19.3232	+1.4526
Pos Tagging	7	TokMem	0.8119		10.5509	
		Ours	0.8810	+0.0690	13.6336	+3.0826
Story Composition	7	TokMem	0.8095		24.0610	
		Ours	0.8286	+0.0190	27.1569	+3.0959
Ethics Classification	6	TokMem	0.9889		55.6402	
		Ours	0.9500	-0.0389	51.8811	-3.7591

Table 6: Ablation study on the effectiveness of <EOC> and TCRA in terms of F1.

<EOC>	TCRA	Tool Selection				Argument			
		2 calls	3 calls	4 calls	Avg.	2 calls	3 calls	4 calls	Avg.
Llama 3.2 1B									
		82.0	80.9	76.0	81.2	67.6	66.0	55.7	66.2
✓		84.4	85.7	76.0	84.6	70.7	72.0	57.0	70.7
✓	✓	89.8	90.3	83.3	89.7	74.8	74.6	65.6	74.3
Llama 3.2 3B									
		77.6	77.7	74.0	77.4	62.9	61.7	56.6	62.0
✓		76.5	79.3	79.1	78.0	59.9	63.0	59.4	61.3
✓	✓	81.0	83.6	82.9	82.4	63.4	67.7	64.6	65.5
Llama 3.1 8B									
		84.3	85.1	72.7	84.1	72.7	70.7	62.5	71.2
✓		86.9	85.6	77.7	85.8	74.6	73.5	68.2	73.8
✓	✓	87.9	86.4	78.5	86.7	76.5	73.7	68.3	74.7

Table 7: Comparison between TCRA and the replace-head experiment.

Backbone	Method	Tool F1	Arguments F1
Llama 3.2 1B	TCRA	89.7	74.3
	Replace	89.9	73.9
Llama 3.2 3B	TCRA	82.4	65.5
	Replace	80.3	61.0

Table 8: Trainable parameter counts in the compositional recall experiment.

Backbone	Fine-tuning	TokMem	Ours
Llama 3.2 1B	0.85M	0.10M	0.21M
Llama 3.2 3B	2.29M	0.15M	0.31M
Llama 3.1 8B	3.41M	0.20M	0.41M

Table 9: Tools used in the experiment of compositional recall for adaptation phase. Tool ID, tool name, and natural-language description are listed.

ID	Tool	Description
1	calculate investment return	Calculates the return on investment based on the initial deposit amount, annual interest rate, and investment period.
2	draw cards	Shuffles a standard deck of 52 cards and draws a specified number of cards from the top.
3	average	Calculates the arithmetic mean of a list of numbers.
4	calculate angle	Calculates the angle between the hour and minute hands of a clock.
5	mean confidence interval	Calculates the confidence interval for a sample mean.
6	find majority element	Finds the majority element in a list, which appears more than $\lfloor n/2 \rfloor$ times.
7	california alimony	Calculates the total alimony one spouse would have to pay to the other in California over a given duration.
8	is sum of cubes	Checks if a number is the sum of the cubes of its digits.
9	find pairs with sum	Finds all pairs of integers in a list that sum up to a target value.
10	bacterial growth	Calculates the bacterial population after a given time based on the initial population and growth rate.
11	get ip zipcode	Retrieves the ZIP code of a given IP address using the ip-api.com API.
12	project investment growth	Projects the growth of an investment over a specified number of years.
13	triangle area	Computes the area of a triangle given its base and height.
14	neuronal activity rate	Calculates the neuronal activity rate based on the synaptic input rate, synaptic weight, and decay constant.
15	find n largest numbers	Finds the n largest numbers in a list.
16	min meeting rooms	Finds the minimum number of meeting rooms required to accommodate all meetings.
17	circle area	Computes the area of a circle given its radius.
18	predict forest area	Predicts the change in forest area over a specified number of years.
19	sort numbers	Sorts a list of numbers in ascending or descending order.
20	is valid email	Checks if an email address is valid.
21	chi square independence test	Performs a Chi-Square test for independence on a 2x2 contingency table.
22	is perfect square	Checks if a number is a perfect square.
23	std deviation	Calculates the standard deviation of a list of numbers.
24	euclidean distance	Computes the Euclidean distance between two points in a 2D plane.
25	find next greater element	Finds the next greater element for each element in a list.
26	calculate quarterly dividend	Calculates the quarterly dividend per share based on the total dividend payout and the number of outstanding shares.
27	loginuser	Logs in a user using the provided username and password, utilizing the Toolbench RapidAPI key for authorization.
28	calculate standard deviation	Calculates the standard deviation of a list of numbers.
29	assess diabetes risk	Assesses the risk of developing type 2 diabetes based on body mass index (BMI) and physical activity level.
30	future investment value	Calculates the future value of an investment based on the present value, annual interest rate, and number of years.
31	numerical derivative	Estimates the derivative of a function at a given point using the central difference formula.
32	batting average	Calculates the batting average of a baseball player based on the number of hits and at-bats.
33	predict evolution rate	Predict the evolutionary rate for a specific species for a given timeframe.
34	get ip location	Retrieves the latitude and longitude coordinates of a given IP address using the ip-api.com API.
35	analyze social group dynamics	Analyzes the social dynamics within a group based on the personality traits and group size.
36	is hotel available	Checks the availability of a hotel for a given date range.
37	compound interest	Calculates the compound interest of an investment.
38	simulate query database	Simulates querying a database based on certain conditions.
39	get order	Retrieves the details of an order by making a GET request to the RapidAPI store.
40	calculate calorie intake	Calculates the recommended daily calorie intake and macronutrient distribution based on personal characteristics and goals.
41	dice roll probability	Calculates the probability of rolling a specific sum with a given number of dice, each having a certain number of faces.
42	linear regression prediction	Predicts the y-value for a given x-value using linear regression.
43	find kth smallest number	Finds the kth smallest number in a list.
44	is power of two	Checks if a number is a power of two.
45	least common multiple	Computes the least common multiple (LCM) of two positive integers.
46	cell density	Calculates the cell density based on the optical density (OD) and dilution factor.
47	independent samples t test	Conducts a two-sample independent t-test and returns the t-statistic, p-value, and conclusion.
48	calculate distance	Calculates the Euclidean distance between two points in n-dimensional space.
49	calculate electric field	Calculate the electric field produced by a charge at a certain distance.
50	structural analysis	Performs structural analysis on a building based on its ID, floor numbers, and analysis mode.

Table 10: Tools used in the experiment of compositional recall across all methods. Tool ID, tool name, and natural-language description are listed.

ID	Tool	Description
51	get products in category	Fetches a list of products from a specified category in a store with pagination.
52	is valid sudoku	Checks if a 9x9 Sudoku board is valid.
53	find minimum rotated sorted array	Finds the minimum element in a rotated sorted array.
54	note duration	Calculates the duration between two musical notes based on their frequencies and the tempo.
55	polygon area shoelace	Calculates the area of a polygon using the shoelace formula.
56	greatest common divisor	Computes the greatest common divisor (GCD) of two non-negative integers.
57	is rotation	Checks if one string is a rotation of another string.
58	generate password	Generates a random password of specified length and character types.
59	calculate grade	Calculates the weighted average grade based on scores and their corresponding weights.
60	reverse string	Reverses the characters in a string.
61	is valid ip address	Checks if a string is a valid IP address (IPv4).
62	is leap year	Checks if a year is a leap year.
63	project population	Projects the population size after a specified number of years.
64	final velocity	Calculates the final velocity of an object given its initial velocity, acceleration, and time.
65	create histogram	Create a histogram based on provided data.
66	cagr	Calculates the Compound Annual Growth Rate (CAGR) of an investment.
67	fibonacci	Calculates the nth Fibonacci number.
68	is power	Checks if a number is a power of a given base.
69	whois	Queries a WhoIS lookup service to retrieve data about a specified domain.
70	whole foods order	Places an order at Whole Foods.
71	get product	Fetches a product's information from the API using the given product ID.
72	find longest word	Finds the longest word in a list of words.
73	binary search	Performs binary search on a sorted list to find the index of a target value.
74	count bits	Counts the number of set bits (1's) in the binary representation of a number.
75	calculate median	Calculates the median of a list of numbers.
76	find first non repeating char	Finds the first non-repeating character in a string.
77	get city from zipcode	Retrieves the city name for a given ZIP code using the Ziptastic API.
78	place safeway order	Order specified items from a Safeway location.
79	potential energy	Calculates the electrostatic potential energy given the charge and voltage.
80	find equilibrium index	Finds the equilibrium index of a list, where the sum of elements on the left is equal to the sum of elements on the right.
81	get pokemon move info	Retrieves information about a Pokémon's move using the PokéAPI.
82	wire resistance	Calculates the resistance of a wire based on its length, cross-sectional area, and material resistivity.
83	find duplicate number	Finds the duplicate number in a list of integers.
84	calculate factorial	Calculates the factorial of a non-negative integer.
85	can attend all meetings	Determines if a person can attend all meetings given a list of meeting time intervals.
86	is palindrome	Checks if a string is a palindrome.
87	generate random string	Generates a random string of specified length and character types.
88	find max subarray sum	Finds the maximum sum of a contiguous subarray within a list of integers.
89	flatten list	Flattens a nested list into a single-level list.
90	format date	Converts a date string from one format to another.
91	directions between 2 locations	Fetches route information between two geographical locations, including distance and duration.
92	cosine similarity	Calculates the cosine similarity between two vectors.
93	binary addition	Adds two binary numbers and returns the result as a binary string.
94	trapezoidal integration	Calculates the definite integral of a function using the trapezoidal rule.
95	solve quadratic	Computes the roots of a quadratic equation given its coefficients.
96	is valid palindrome	Checks if a string is a valid palindrome, considering only alphanumeric characters and ignoring case.
97	integrate	Calculate the area under a curve for a specified function between two x values.
98	monthly mortgage payment	Calculates the monthly mortgage payment based on the loan amount, annual interest rate, and loan term.
99	investment profit	Calculates the profit from an investment based on the initial amount, annual return rate, and time.
100	auto complete	Fetches auto-complete suggestions for a given word using an external API.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.